Table 3.2 Examples of container metrics

Category	Example metric in cAdvisor
CPU	`container_cpu_usage_seconds_total` (cumulative CPU time consumed) and `container_cpu_load_average_10s` (container CPU load average over the past 10 seconds)
Memory	`container_memory_max_usage_bytes` (maximum app memory usage) and `machine_memory_bytes` (memory installed on the host)
File system	`container_file_descriptors` (open file descriptors for the container)
Network	`container_network_receive_packets_total` (cumulative count of packets received)

As a developer, you want to have access to the application-level logs, and you will likely find certain metrics useful, be that to optimize your app in terms of resource usage or for troubleshooting. For operators, having a flexible way to configure log routing is important (requirements of where to send certain types of logs and/or retention may change) as well as being able to use the metrics the runtime exposes to ensure a reliable and efficient usage of the host. Now that we have an idea about the container runtime and application basic signal sources, let's move on to how containers are usually used at scale: a container orchestrator, with Kubernetes being the most popular and widely used example.

3.2.3 Kubernetes

From an observability standpoint, Kubernetes is both advanced and limited. To understand this statement, let's step back a bit: on the one hand, Kubernetes defines several APIs and exposes a range of signals; on the other hand, it assumes that you take care of the telemetry and processing parts. That is, without a comprehensive strategy on how to collect, route, store, and query the signals Kubernetes offers, there is little that is available out of the box for you to consume signals.

Figure 3.2 provides a high-level overview of the signal sources available in Kubernetes (https://kubernetes.io/docs/concepts/architecture/), both in the control plane as well as the data plane.

The Kubernetes control plane manages the state of all resources in the cluster, such as workloads, namespaces, nodes, and so on, and allows you to query and change the state of the resources under management. In the data plane, several of omponents work together to launch, health check, report, and terminate the workloads (containers running in pods). Note that while Kubernetes itself comes with the promise of portable workloads, the low-level bits (from load balancers to networked storage) can and do differ across cloud providers, including APIs and formats.

The logs and metrics signal types are already well supported in Kubernetes, with logs (http://mng.bz/8rvK) being more distributed over various places and the community